

Que1:

1. modify by removing sema logic && observe by running multiple times

Code:

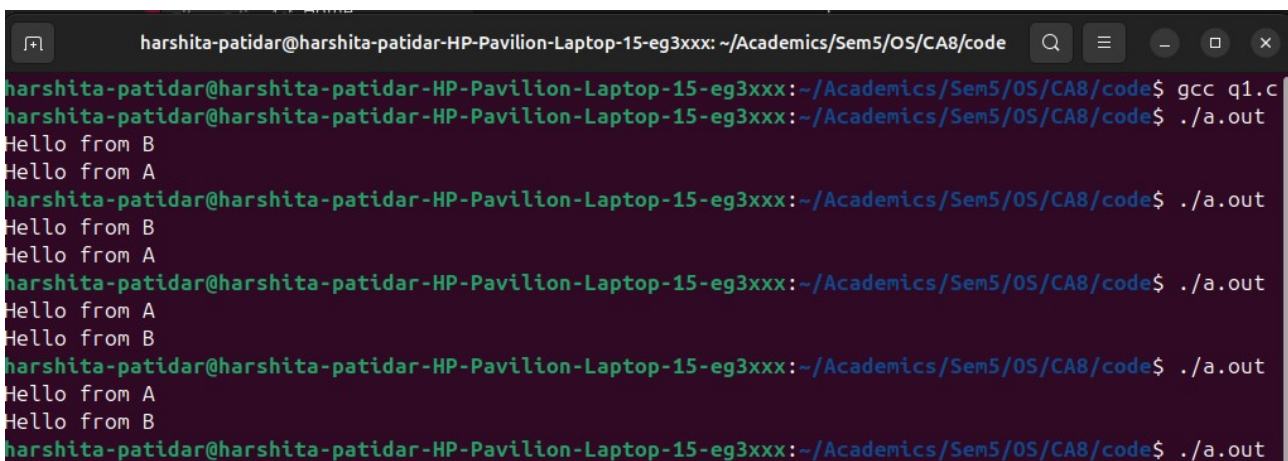
```
#include <stdio.h>
#include <pthread.h>

void* threadA(void* arg)
{
    printf("Hello from A\n");
    return NULL;
}
void* threadB(void* arg)
{
    printf("Hello from B\n");
    return NULL;
}
int main()
{
    pthread_t tA, tB;

    pthread_create(&tA, NULL, threadA, NULL);
    pthread_create(&tB, NULL, threadB, NULL);

    pthread_join(tA, NULL);
    pthread_join(tB, NULL);

    return 0;
}
```



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ gcc q1.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from B
Hello from A
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from B
Hello from A
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from A
Hello from B
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from A
Hello from B
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ ./a.out
```

2.explain why order changes

Without semaphores, the threads are unsynchronized, creating a race condition. scheduler decides which thread runs first, leading to an unpredictable output order upon multiple executions.

3. add one more thread C which should only execute only after thread b

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t sem1, sem2;

void* threadA(void* arg)
{
    printf("Hello from A\n");
    sem_post(&sem1);
    return NULL;
}
void* threadB(void* arg)
{
    sem_wait(&sem1);
    printf("Hello from B\n");
    sem_post(&sem2);
    return NULL;
}
void* threadC(void* arg)
{
    sem_wait(&sem2);
    printf("Hello from C\n");
    return NULL;
}
int main()
{
    pthread_t tA, tB, tC;

    sem_init(&sem1, 0, 0);
    sem_init(&sem2, 0, 0);

    pthread_create(&tA, NULL, threadA, NULL);
    pthread_create(&tB, NULL, threadB, NULL);
    pthread_create(&tC, NULL, threadC, NULL);
    pthread_join(tA, NULL);
    pthread_join(tB, NULL);
    pthread_join(tC, NULL);

    sem_destroy(&sem1);
    sem_destroy(&sem2);

    return 0;
}
```

```
}
```

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ gcc q1.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from A
Hello from B
Hello from C
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from A
Hello from B
Hello from C
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Hello from A
Hello from B
Hello from C
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$
```

Observations:

two semaphores create a dependency chain, acting like signals in a relay race . Thread B is forced to wait for Thread A to finish, and Thread C is forced to wait for Thread B to finish, guaranteeing the execution order is always A -> B -> C.

Que 2: Rendezvous

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

sem_t aArrived,bArrived;

void* threadA(void* arg)
{
    printf("A1 (Starting task) \n");
    sem_post(&aArrived);
    sem_wait(&bArrived);
    printf("A2 (Ending task) \n");
    return NULL;
}
void* threadB(void* arg)
{
    printf("B1 (Starting task)\n");
    sem_post(&bArrived);
    sem_wait(&aArrived);
    printf("B2 (Ending task) \n");
    return NULL;
}
int main()
{
    pthread_t tA, tB;

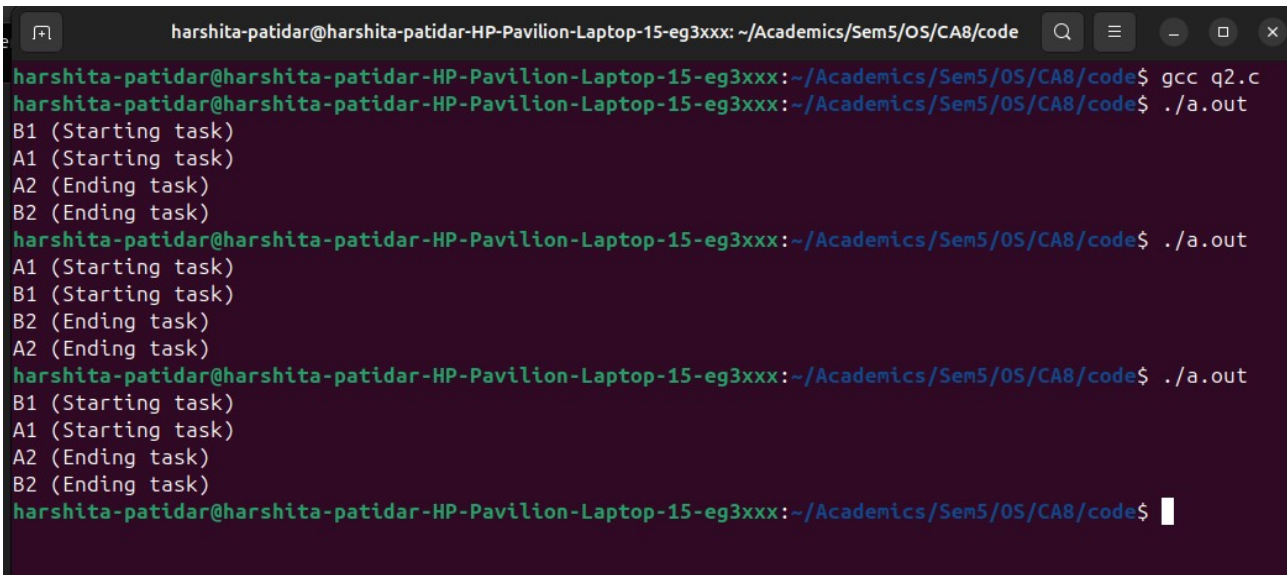
    sem_init(&aArrived, 0, 0);
```

```
sem_init(&bArrived, 0, 0);

pthread_create(&tA, NULL, threadA, NULL);
pthread_create(&tB, NULL, threadB, NULL);

pthread_join(tA, NULL);
pthread_join(tB, NULL);

sem_destroy(&aArrived);
sem_destroy(&bArrived);
return 0;
```



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ gcc q2.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
B1 (Starting task)
A1 (Starting task)
A2 (Ending task)
B2 (Ending task)
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
A1 (Starting task)
B1 (Starting task)
B2 (Ending task)
A2 (Ending task)
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
B1 (Starting task)
A1 (Starting task)
A2 (Ending task)
B2 (Ending task)
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$
```

Observations:

This code creates a rendezvous point, where each thread signals its arrival (`sem_post`) before waiting for the other to arrive (`sem_wait`). This ensures that both threads print their "Starting task" message before either can print its "Ending task" message, successfully avoiding a deadlock.

QUE3: Thread Synchronization Using Mutex

implement counter upto 1000000 with 10 thread

Part 1: Without Mutex:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;

void* increment(void* arg)
{
    for(int i = 0; i < 100000; i++)
    {
        counter = counter + 1;
    }
    return NULL;
}

int main()
{
    pthread_t t[10];

    for(int i=0;i<10;i++)
    {
        pthread_create(&t[i], NULL, increment, NULL);
    }

    for(int i=0;i<10;i++)
    {
        pthread_join(t[i], NULL);
    }

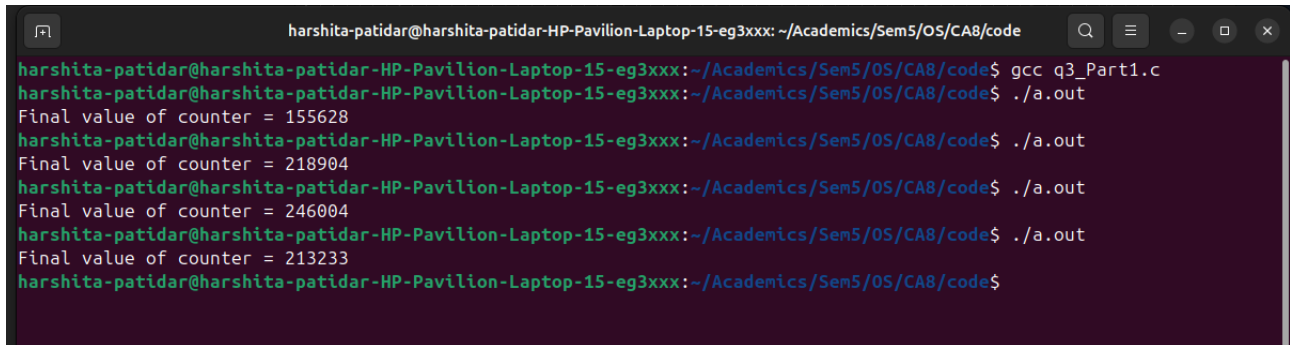
    printf("Final value of counter = %d\n",counter);

    return 0;
}
```

```
}
```

Race Condition observation:

Screenshot:



```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ gcc q3_Part1.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 155628
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 218904
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 246004
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 213233
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$
```

Part 2: With Mutex

Code:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;
void* increment(void* arg)
{
    for(int i = 0; i < 100000; i++)
    {
        pthread_mutex_lock(&lock);
        counter = counter + 1;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t[10];
    pthread_mutex_init(&lock, NULL);

    for(int i=0;i<10;i++)
    {
        pthread_create(&t[i], NULL, increment, NULL);
    }

    for(int i=0;i<10;i++)
    {
```

```

    pthread_join(t[i], NULL);
}

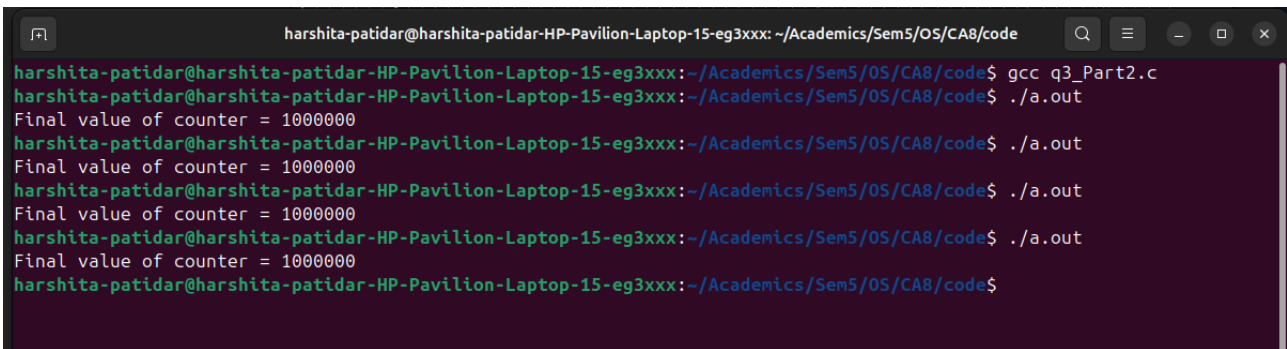
printf("Final value of counter = %d\n",counter);

return 0;
}

```

Why race condition occurs without mutex and not with mutex?:

A race condition occurs without a mutex because threads read and write to the shared counter simultaneously, causing some increments to be overwritten and lost. A mutex prevents this by acting as a lock, forcing threads to take turns and ensuring the increment operation is atomic, which guarantees a correct final result.



```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ gcc q3_Part2.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 1000000
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 1000000
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 1000000
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Final value of counter = 1000000
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$

```

Que 4: Multiplex / Bounded Semaphore

Explain how the semaphore controls concurrent section simultaneously:

semaphore is initialized with a counter= number of allowed threads (3). To enter the critical section, a thread must decrement this counter using `sem_wait`. If the counter is zero, `sem_wait` blocks the thread, forcing it to wait until another thread exits and increments the counter with `sem_post`, thereby strictly enforcing the limit.

Que 5: Barrier Synchronization

Part A: Deadlock

code:

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

```

```

#define N 5

```

```

int count = 0;
pthread_mutex_t mutex;

```

```

sem_t barrier;

void* thread_func(void* arg)
{
    int id = *(int*)arg;
    printf("Thread %d reached the barrier\n", id);
    pthread_mutex_lock(&mutex);
    count++;
    if (count == N)
    {
        sem_post(&barrier); // Only one thread is signaled!
    }
    pthread_mutex_unlock(&mutex);
    sem_wait(&barrier); // All threads wait, but only one is signaled
    printf("Thread %d passed the barrier\n", id);
    return NULL;
}

int main() {
    pthread_t threads[N];
    int ids[N];
    pthread_mutex_init(&mutex, NULL);
    sem_init(&barrier, 0, 0);
    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, thread_func, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }
    pthread_mutex_destroy(&mutex);
    sem_destroy(&barrier);
    return 0;
}

```

Why deadlock happen?

In first code deadlocks because the last thread to arrive signals the semaphore only **once**, allowing just one of the n waiting threads to pass. The other n-1 threads remain stuck at sem_wait forever, as no more signals are ever sent.


```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code$ gcc q5_Part1.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Thread 1 reached the barrier
Thread 2 reached the barrier
Thread 3 reached the barrier
Thread 4 reached the barrier
Thread 5 reached the barrier
Thread 5 passed the barrier
```

Part B: Reusable Barrier

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define N 5

int count = 0;
pthread_mutex_t mutex;
sem_t turnstile1, turnstile2;

void* thread_func(void* arg)
{
    int id = *(int*)arg;
    printf("Thread %d reached the barrier\n", id);
    pthread_mutex_lock(&mutex);
    count++;
    if (count == N)
    {
        sem_post(&turnstile1);
    }
    pthread_mutex_unlock(&mutex);
    sem_wait(&turnstile1);
    sem_post(&turnstile1);
    printf("Thread %d passed the barrier\n", id);

    pthread_mutex_lock(&mutex);
    count--;

    if(count==0)
    {
        sem_post(&turnstile2);
    }
    pthread_mutex_unlock(&mutex);
    sem_wait(&turnstile2);
    sem_post(&turnstile2);
    return NULL;
}
```

```

}

int main() {
    pthread_t threads[N];
    int ids[N];
    pthread_mutex_init(&mutex, NULL);
    sem_init(&turnstile1, 0, 0);
    sem_init(&turnstile2, 0, 0);
    for (int i = 0; i < N; i++) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, thread_func, &ids[i]);
    }
    for (int i = 0; i < N; i++) {
        pthread_join(threads[i], NULL);
    }
    pthread_mutex_destroy(&mutex);
    sem_destroy(&turnstile1);
    sem_destroy(&turnstile2);

    printf("ALL THREADS FINISHED!\n");
    return 0;
}

```

The screenshot shows a terminal window with the following output:

```

harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/CA8/code
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ gcc q5_Part2.c
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Thread 2 reached the barrier
Thread 4 reached the barrier
Thread 1 reached the barrier
Thread 5 reached the barrier
Thread 3 reached the barrier
Thread 3 passed the barrier
Thread 2 passed the barrier
Thread 4 passed the barrier
Thread 1 passed the barrier
Thread 5 passed the barrier
ALL THREADS FINISHED!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$ ./a.out
Thread 4 reached the barrier
Thread 1 reached the barrier
Thread 3 reached the barrier
Thread 2 reached the barrier
Thread 5 reached the barrier
Thread 5 passed the barrier
Thread 4 passed the barrier
Thread 1 passed the barrier
Thread 3 passed the barrier
Thread 2 passed the barrier
ALL THREADS FINISHED!
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/CA8/code$

```

Observation:

reusable barrier solves this by creating a chain reaction. The last thread opens the first gate (turnstile1), and each thread that passes through immediately opens it for the next one, ensuring all threads can proceed. A second gate turnstile2 is used to safely reset the barrier, preventing race conditions and making it reusable.